

LAB ASSIGNMENT-6

1. Implement a class ***Library*** that contains **an array of *Book* objects**. The ***Book*** class should have attributes like **title, author, and ISBN**. The ***Library*** class should provide following methods:
 - a. **bool addNewBook(title,author,ISBN)** to add the new book in the book list. The return type should be Boolean to check whether the book is added or not. This function should take the argument using pass-by-reference method. The attribute of the class (title, author, and ISBN) and the function arguments of function **addNewBook** should be same. Use the **scope resolution operator** to access the class data members.
 - b. **bool removeBooks(ISBN)** to remove the book using the input parameter ISBN. The return type should be Boolean to check whether the book is removed or not. This function also should take the argument using pass-byreference method. Define function outside the class using scope resolution operator.
 - c. **displayDetails()** to list the books.

Write a main function to demonstrate adding, removing, and displaying books from the library. Add at least 5 books to demonstrate the adding book, 1 book to remove from the list.

Code:

```
#include <iostream>
#include <string>
using namespace std;
class Book
{
public:
    string title;
    string author;
    string ISBN;
};
class Library
{
private:
```

```

    Book books[10];

    int count = 0;

public:
    bool addNewBook(string &title, string &author, string &ISBN);
    bool removeBooks(string &ISBN);
    void displayDetails();
};

bool Library::addNewBook(string &title, string &author, string &ISBN)
{
    if(count >= 10)
    {
        return false;
    }
    books[count].title = title;
    books[count].author = author;
    books[count].ISBN = ISBN;
    count++;

    return true;
}

bool Library::removeBooks(string &ISBN)
{
    for(int i = 0; i < count; i++)
    {
        if(books[i].ISBN == ISBN)
        {
            for(int j = i; j < count - 1; j++)
            {
                books[j] = books[j + 1];
            }
            count--;
            return true;
        }
    }
}

```

```

    }
}
return false;
}

void Library::displayDetails()
{
    for(int i = 0; i < count; i++)
    {
        cout << "\nTitle: " << books[i].title << endl;
        cout << "Author: " << books[i].author << endl;
        cout << "ISBN: " << books[i].ISBN << endl;
    }
}

int main()
{
    Library lib;

    string t1 = "C++";
    string a1 = "Bjarne";
    string i1 = "101";
    string t2 = "Java";
    string a2 = "James";
    string i2 = "102";
    string t3 = "Python";
    string a3 = "Guido";
    string i3 = "103";
    string t4 = "OS";
    string a4 = "Galvin";
    string i4 = "104";
    string t5 = "DBMS";
    string a5 = "Korth";

```

```

string i5 = "105";

lib.addNewBook(t1,a1,i1);
lib.addNewBook(t2,a2,i2);
lib.addNewBook(t3,a3,i3);
lib.addNewBook(t4,a4,i4);
lib.addNewBook(t5,a5,i5);
lib.displayDetails();
lib.removeBooks(i3);
cout << "After Removing Book"<<endl;
lib.displayDetails();
return 0;
}

```

2. Implement the question 1 using constructor. Do the following modification:
 - a. Add default constructor, parameterized constructor, and copy constructor to add the details of the book using the same parameters, i.e., **title, author, and ISBN**.
 - b. Use the “this” pointer instead of scope resolution operator to access the data members of the class inside the constructor and function definition.
 - c. Create function **bool removeBooks(ISBN)** same as the question 1.
 - d. **displayDetails()** same as question
 - e. Create array of objects using 1._INITIALIZER list, 2. Dynamic initialization

Code:

```

#include <iostream>
#include <string>
using namespace std;
class Book
{
private:
    string title;
    string author;
    string ISBN;
public:
    Book()

```

```

{
    this->title = "None";
    this->author = "None";
    this->ISBN = "000";
}

Book(string title, string author, string ISBN)
{
    this->title = title;
    this->author = author;
    this->ISBN = ISBN;
}

Book(const Book &b)
{
    this->title = b.title;
    this->author = b.author;
    this->ISBN = b.ISBN;
}

bool removeBooks(const string &ISBN)
{
    if(this->ISBN == ISBN)
    {
        this->title = "";
        this->author = "";
        this->ISBN = "";

        return true;
    }
    return false;
}

void displayDetails()
{
    if(ISBN != "")

```

```

        {
            cout << "Title: " << title << endl;
            cout << "Author: " << author << endl;
            cout << "ISBN: " << ISBN << endl;
        }
    }
};

int main()
{
    Book b1("C++","Bjarne","101");
    Book b2("Java","James","102");
    Book *b3 = new Book("Python","Guido","103");
    Book b4 = b1;
    b1.displayDetails();
    b2.displayDetails();
    b3->displayDetails();
    b4.displayDetails();
    b2.removeBooks("102");
    cout << "After Removing Book"<<endl;
    b1.displayDetails();
    b2.displayDetails();
    b3->displayDetails();
    b4.displayDetails();

    delete b3;
    return 0;
}

```

3. Create a class *Account* with attributes *account number (of const long type)*, *transaction ID (of long type)*, *transaction type (credit/debit) (of string type)*, and *balance (of double type)*. The Account class should provide following methods:

- a. **long depositAmount(to, from, amount)**. The “to” and “from” are the account number of the sender and receiver. The function should return the transaction id.
- b. **long creditAmount(to, from, amount)** similar to the depositAmount function. This function should also return the transaction id.
- c. **void displayDetails()** should display all the details such as account number, remaining balance, transaction history (credited/debited). Make this function as the const function.
- d. In all the functions, make all the arguments as the const arguments.

Write a main function to demonstrate all the function calls. Create at least 5 accounts objects (using array or without array) to demonstrate the operations of credit and debit.

Note: Use the scope resolution operator to include in-built libraries (cin, cout, endl, string etc.)

Code:

```
#include <iostream>
#include <string>
using namespace std;
class Account
{
private:
    const long accountNumber;
    long transactionID;
    string transactionType;
    double balance;
    static long nextID;
public:
    Account(long accNo, double bal)
        : accountNumber(accNo)
    {
        balance = bal;
```

```

        transactionID = 0;
        transactionType = "None";
    }
    long depositAmount(const long to,
                       const long from,
                       const double amount)
    {
        if(accountNumber == to)
        {
            balance += amount;
            transactionType = "Credit";
            transactionID = ++nextID;
        }
        return transactionID;
    }
    long creditAmount(const long to,
                      const long from,
                      const double amount)
    {
        if(accountNumber == from)
        {
            balance -= amount;
            transactionType = "Debit";
            transactionID = ++nextID;
        }
        return transactionID;
    }
    void displayDetails() const
    {
        cout << "Account Number: " << accountNumber << endl;
        cout << "Balance: " << balance << endl;
        cout << "Last Transaction: " << transactionType << endl;
    }

```



```

        cout << "Transaction ID: " << transactionID << endl;
    }
};

long Account::nextID = 1000;

int main()
{
    Account a1(101,5000);
    Account a2(102,4000);
    Account a3(103,3000);
    Account a4(104,2000);
    Account a5(105,1000);

    a1.creditAmount(102,101,500);
    a2.depositAmount(102,101,500);
    a3.creditAmount(104,103,700);
    a4.depositAmount(104,103,700);
    a1.displayDetails();
    a2.displayDetails();
    a3.displayDetails();
    a4.displayDetails();
    a5.displayDetails();

    return 0;
}

```

4. Write a program to add data objects of two different classes using friend functions.

Code:

```

#include <iostream>

using namespace std;

class B;

class A
{
private:
    int x;

```

```

public:
    A(int a)
    {
        x = a;
    }
    friend int add(A,B);
};

class B
{
private:
    int y;
public:
    B(int b)
    {
        y = b;
    }
    friend int add(A,B);
};

int add(A obj1, B obj2)
{
    return obj1.x + obj2.y;
}

int main()
{
    A a(10);
    B b(20);

    cout << "Sum = "
        << add(a,b);

    return 0;
}

```

5. Define a class named ***Complex*** with properties (real and imaginary) and methods as per following details.

- a. ***Parameterized constructor*** and ***copy constructor*** to initialize object values.
- b. ***void display ()*** to display complex number.
- c. ***void sum (Complex, Complex)*** to add two complex numbers (objects of Complex class). Implement this function as the friend function.

Code:

```
#include <iostream>

using namespace std;

class Complex
{
private:
    int real;
    int imaginary;
public:
    Complex(int r, int i)
    {
        real = r;
        imaginary = i;
    }
    Complex(const Complex &c)
    {
        real = c.real;
        imaginary = c.imaginary;
    }
    void display()
    {
        cout << real
            << " + "
            << imaginary
            << "i" << endl;
    }
    friend void sum(Complex, Complex);
```

```

};

void sum(Complex c1, Complex c2)
{
    int r = c1.real + c2.real;
    int i = c1.imaginary + c2.imaginary;
    cout << "Sum = "
        << r
        << " + "
        << i
        << "i";
}

int main()
{
    Complex c1(2,3);
    Complex c2(4,5);
    Complex c3 = c1;
    cout << "Complex Number 1: ";
    c1.display();
    cout << "Complex Number 2: ";
    c2.display();
    cout << "Copied Complex Number: ";
    c3.display();
    sum(c1,c2);
    return 0;
}

```